CITED REFERENCES AND FURTHER READING:

Felsenstein, J. 2004, *Inferring Phylogenies* (Sunderland, MA: Sinauer).[1]

Warnow, T. 1996, "Some Combinatorial Optimization Problems in Phylogenetics," in *Proceedings of the International Colloquium on Combinatorics and Graph Theory*, eds. A. Gyarfas, L. Lovasz, L.A. Szekely, Bolyai Society Mathematical Studies, vol. 7, (Budapest: Bolyai Society).[2]

Agarwala, R. et al. 1999, "On the Approximability of Numerical Taxonomy," *SIAM Journal of Computing*, vol. 28, no. 3, pp. 1073-1085.[3]

Cohen, J. and Farach M. 1997, in *Proceedings of the 8th Annual ACM-SIAM Symposium on Discrete Algorithms SODA '97* (Philadelphia: S.I.A.M.).[4]

Rice, K. and Warnow, T. 1997, "Parsimony Is Hard to Beat," in *Computing and Combinatorics*, 3rd Annual International Conference COCOON '97, T. Jiang and D.T. Lee, eds. (New York: Springer).[5]

Saitou, N. and Nei, M. 1987, "The Neighbor-joining Method: A New Method for Reconstructing Phylogenetic Trees," *Molecular Biology and Evolution*, vol. 4, no. 4, pp. 406-425; see also *op. cit.*, vol. 5, no. 6, pp. 729-731.[6]

Swofford, D. 2004+, *PAUP: Phylogenetic Analysis Using Parsimony and Other Methods*, version 4, at `http://paup.csit.fsu.edu/`.[7]

Felsenstein, J. 2003+, *PHYLIP: Phylogeny Inference Package*, version 3.6, at `http://evolution.genetics.washington.edu/phylip.html`.[8]

Hall, B.G. 2004, *Phylogenetic Trees Made Easy: A How-To Manual*, 2nd ed. (Sunderland, MA: Sinauer).[9]

Numerical Recipes Software 2007, "Code for Rendering a Phylagglom Tree in Simple PostScript," *Numerical Recipes Webnote No. 28*, at `http://www.nr.com/webnotes?28` [10]

Strimmer, K. and von Haeseler, A. 1996, "Quartet Puzzling: A Quartet Maximum Likelihood Method for Reconstructing Tree Topologies," *Molecular Biology and Evolution*, vol. 13, no. 7, pp. 964-969.[11]

Goloboff, P.A. 1999, "Analyzing Large Data Sets in Reasonable Times: Solutions for Composite Optima," *Cladistics*, vol. 15, pp. 415-428; also see `http://www.cladistics.com`.[12]

Nixon, K.C. 1999, "The Parsimony Ratchet, a New Method for Rapid Parsimony Analysis," *Cladistics*, vol. 15, pp. 407-414.[13]

16.5 Support Vector Machines

The *support vector machine* or *SVM*, first described by Vapnik and collaborators in 1992 [1], has rapidly established itself as a powerful algorithmic approach to the problem of *classification* within the larger context known as *supervised learning*. SVMs are no more "machines" than are Turing "machines"; the use of the word is inherited from that part of computer science long known as "machine learning." A number of classification problems whose solutions were previously dominated by neural nets and more complicated methods have been found to be straightforwardly solvable by SVMs [2]. Moreover, SVMs are generally easier to implement than are neural nets; and it is generally easier to intuit what SVMs "think they are doing" than for neural nets, which are famous for their opaqueness.

In the supervised learning problem of classification, we are given a set of *training data* consisting of m points,

$$(\mathbf{x}_i, y_i) \qquad i = 1, \ldots, m \tag{16.5.1}$$

Each $\mathbf{x}_i$ is a *feature vector* in n dimensions (say) that describes the data point, while each corresponding y_i has the value ± 1, indicating whether that data point is in $(+1)$ or out of (-1) the set that we want to learn to recognize. We desire a *decision rule*, in the form of a function $f(\mathbf{x})$ whose sign predicts the value of y, not just for the data in the training set, but also for new values of $\mathbf{x}$ never before seen.

For some applications, the feature vector $\mathbf{x}$ truly lives in the continuous space $\mathbf{R}^n$. However, you are allowed to be creative in mapping your problem into this framework: In many applications, the feature vector will be a binary vector that encodes the presence or absence of many "features" (hence its name). For example, the feature vector describing a DNA sequence of length p could have $n = 4p$ dimensions, with each base position using four dimensions, and having the value one in one of the four (depending on whether it is A, C, G, or T), zero in the others.

16.5.1 Special Case of Linearly Separable Data

One can understand SVMs conceptually as a series of generalizations from an idealized, and rather unrealistic, starting point, We discuss these generalizations sequentially in the rest of this section. The starting point is the special case of *linearly separable data*. In this case, we are told (by an oracle?) that there exists a hyperplane in n dimensions, that is, an $n-1$ dimensional surface defined by the equation

$$f(\mathbf{x}) \equiv \mathbf{w} \cdot \mathbf{x} + b = 0 \tag{16.5.2}$$

that completely separates the training data. In other words, all the training points with $y_i = 1$ lie on one side of the hyperplane (and thus have $f(\mathbf{x}_i) > 0$), while all the training points with $y_i = -1$ lie on the other side (and have $f(\mathbf{x}_i) < 0$). All we have to do is find $\mathbf{w}$ (a normal vector to the hyperplane) and b (an offset). Then $f(\mathbf{x})$ in equation (16.5.2) will be the decision rule.

Actually, we can do better than this. In general, more than one hyperplane will separate linearly separable data. Let's pick the hyperplane that has the largest *margin*, i.e., maximizes the perpendicular distance to points nearest to the hyperplane on both sides. Specifically, given any hyperplane that separates the data, we can always scale $\mathbf{w}$ by a constant and adjust b appropriately, to make

$$\begin{aligned} \mathbf{w} \cdot \mathbf{x}_i + b &\geq +1 \quad \text{when } y_i = +1 \\ \mathbf{w} \cdot \mathbf{x}_i + b &\leq -1 \quad \text{when } y_i = -1 \end{aligned} \tag{16.5.3}$$

These equations represent parallel bounding hyperplanes that separate the data (see Figure 16.5.1), a structure whimsically called a *fat plane*. With a bit of analytical geometry, you can easily convince yourself that the perpendicular distance between the bounding hyperplanes (twice the margin) is

$$2 \times \text{margin} = 2(\mathbf{w} \cdot \mathbf{w})^{-1/2} \tag{16.5.4}$$

Also note that both cases of equation (16.5.3) can be summarized as the single equation

$$y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1 \tag{16.5.5}$$

What we see is that the fattest fat plane, also known as the *maximum margin SVM*, can be found by solving a particular problem in quadratic programming:

$$\begin{array}{ll} \text{minimize:} & \frac{1}{2}\mathbf{w} \cdot \mathbf{w} \\ \text{subject to:} & y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1 \qquad i = 1, \ldots, m \end{array} \tag{16.5.6}$$

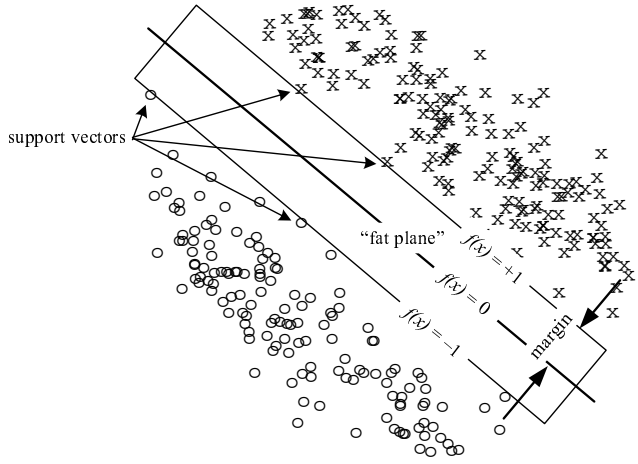

Figure 16.5.1. Support vector machine (SVM) in the idealized case of linearly separable data. We want to classify regions of the plane as containing x's or o's. The "fat plane" defined by $-1 \le f(x) \le 1$ is chosen to maximize the margin (shown). At such a maximum, a small number of points, the "support vectors," will lie on the bounding planes.

Note that we minimize $\mathbf{w} \cdot \mathbf{w}$ instead of equivalently maximizing its reciprocal. The factor of $1/2$ merely simplifies some algebra, later.

General methods for solving quadratic programming problems like the above are discussed in [3,4]. Later in this section, we discuss a method specialized for some SVMs. For now, consider the solution of (16.5.6) and similar problems as an available "black box."

At a solution of (16.5.6), some (usually a small number) of the data points must lie exactly on one or the other bounding hyperplane, because, otherwise, the fat plane could have been made fatter. These data points, with $f(\mathbf{x}) = \pm 1$, are called the *support vectors* of the solution. However, despite the fact that support vector machines were originally named after these support vectors, they don't play much of a role in the more realistic generalizations that we will soon discuss.

16.5.2 Primal and Dual Problems in Quadratic Programming

The first of our promised generalizations may at first sight seem a puzzling direction to go, since it consists merely of replacing one quadratic programming problem with another. We will see later, however, that this replacement has profound consequences.

The general problem in quadratic programming, known as the *primal* problem, can be stated as

$$\begin{array}{ll} \text{minimize:} & f(\mathbf{w}) \\ \text{subject to:} & g_j(\mathbf{w}) \le 0 \\ & h_k(\mathbf{w}) = 0 \end{array} \tag{16.5.7}$$

where $f(\mathbf{w})$ is quadratic in $\mathbf{w}$; $g(\mathbf{w})$ and $h(\mathbf{w})$ are affine in $\mathbf{w}$ (i.e., linear plus a constant); and j and k index, respectively, the sets of inequality and equality constraints.

Every primal problem has a *dual* problem, which can be thought of as an alternative way of solving the primal problem (cf. §10.11.1). To get from the primal to the dual, one writes a Lagrangian that incorporates both the quadratic form, and — with Lagrange multipliers — all the constraints, namely,

$$\mathcal{L} \equiv \tfrac{1}{2} f(\mathbf{w}) + \sum_j \alpha_j g_j(\mathbf{w}) + \sum_k \beta_k h_k(\mathbf{w}) \tag{16.5.8}$$

One then writes this subset of conditions for an extremum:

$$\frac{\partial \mathcal{L}}{\partial w_i} = 0, \qquad \frac{\partial \mathcal{L}}{\partial \beta_k} = 0, \tag{16.5.9}$$

and uses the resulting equations algebraically to eliminate $\mathbf{w}$ from $\mathcal{L}$, in favor of $\boldsymbol{\alpha}$ and $\boldsymbol{\beta}$ (where we now write the α_j's and β_k's as vectors). Call the result, the reduced Lagrangian, $\mathcal{L}(\boldsymbol{\alpha}, \boldsymbol{\beta})$. Then the important result, which follows from the so-called *strong duality* and *Kuhn-Tucker* theorems (cf. §10.11.1), is that the solution of the following dual problem is equivalent to the original primal problem:

maximize:	$\mathcal{L}(\boldsymbol{\alpha}, \boldsymbol{\beta})$	
subject to:	$\alpha_j \geq 0$	for all j

(16.5.10)

In fact, this result is more general than quadratic programming and is true, roughly speaking, for any convex $f(\mathbf{w})$. Furthermore, if $\hat{\mathbf{w}}$ is the optimal solution of the primal problem, and $\hat{\boldsymbol{\alpha}}, \hat{\boldsymbol{\beta}}$ are the optimal solutions of the dual problem, we have

$$\begin{aligned} f(\hat{\mathbf{w}}) &= \mathcal{L}(\hat{\boldsymbol{\alpha}}, \hat{\boldsymbol{\beta}}) \\ \hat{\alpha}_j \, g_j(\hat{\mathbf{w}}) &= 0 \qquad \text{for all } j \end{aligned} \tag{16.5.11}$$

The latter condition is called the *Karush-Kuhn-Tucker complementarity condition*. It says that at least one of $\hat{\alpha}_j$ and $g_j(\hat{\mathbf{w}})$ must be zero for each j. (We previously met the linear case in equation 10.11.5.) This means that, from the solution of the dual problem, you can instantly identify inequality constraints in the primal problem that are "pinned" against their limit, namely those with nonzero $\hat{\alpha}_j$'s in the solution of the dual.

16.5.3 Dual Formulation of the Maximum Margin SVM

The above procedure is readily performed on the quadratic programming problem (16.5.6) for the maximum margin SVM. There are no β_k's, since there are no equality constraints. The Lagrangian (16.5.8) is

$$\mathcal{L} = \tfrac{1}{2}\mathbf{w} \cdot \mathbf{w} + \sum_i \alpha_i [1 - y_i(\mathbf{w} \cdot \mathbf{x}_i + b)] \tag{16.5.12}$$

The conditions for an extremum are

$$0 = \frac{\partial \mathcal{L}}{\partial \mathbf{w}} = \mathbf{w} - \sum_i \alpha_i y_i \mathbf{x}_i \quad \Longrightarrow \quad \hat{\mathbf{w}} = \sum_i \hat{\alpha}_i y_i \mathbf{x}_i \tag{16.5.13}$$

and

$$0 = \frac{\partial \mathcal{L}}{\partial b} = \sum_i \alpha_i y_i \tag{16.5.14}$$

Substituting equations (16.5.13) and (16.5.14) back into (16.5.12) gives the reduced Lagrangian

$$\begin{aligned} \mathcal{L}(\boldsymbol{\alpha}) &= \sum_j \alpha_j - \tfrac{1}{2} \sum_{j,k} \alpha_j y_j (\mathbf{x}_j \cdot \mathbf{x}_k) y_k \alpha_k \\ &\equiv \mathbf{e} \cdot \boldsymbol{\alpha} - \tfrac{1}{2} \boldsymbol{\alpha} \cdot \mathrm{diag}(\mathbf{y}) \cdot \mathbf{G} \cdot \mathrm{diag}(\mathbf{y}) \cdot \boldsymbol{\alpha} \end{aligned} \tag{16.5.15}$$

In the second form of the above equation we introduce some convenient matrix notation: $\mathbf{e}$ is the vector whose components are all unity, diag denotes a diagonal matrix formed from a vector in the obvious way, and $\mathbf{G}$ is the *Gram matrix* of dot products of all the $\mathbf{x}_j$'s,

$$G_{ij} \equiv \mathbf{x}_i \cdot \mathbf{x}_j \tag{16.5.16}$$

Remember that subscripts on $\mathbf{x}$ don't indicate components, but rather index which data point is referenced.

The dual problem, in toto, thus turns out to be

minimize:	$\frac{1}{2} \boldsymbol{\alpha} \cdot \mathrm{diag}(\mathbf{y}) \cdot \mathbf{G} \cdot \mathrm{diag}(\mathbf{y}) \cdot \boldsymbol{\alpha} - \mathbf{e} \cdot \boldsymbol{\alpha}$	
subject to:	$\alpha_i > 0$	for all i
	$\boldsymbol{\alpha} \cdot \mathbf{y} = 0$	(from 16.5.14)

(16.5.17)

We also have the Karush-Kuhn-Tucker relation,

$$\hat{\alpha}_i \, [y_i (\hat{\mathbf{w}} \cdot \mathbf{x}_i + b) - 1] = 0 \tag{16.5.18}$$

Equation (16.5.13) tells how to get the optimal solution $\hat{\mathbf{w}}$ of the primal problem from the solution $\hat{\boldsymbol{\alpha}}$ of the dual. Equation (16.5.18) is then used to get $\hat{b}$: Find *any* nonzero α_i, then, with the corresponding y_i, $\mathbf{x}_i$, and $\hat{\mathbf{w}}$, solve the above relation for $\hat{b}$. Alternatively, one can average out some roundoff error by taking a weighted average of α_i's,

$$\hat{b} = \sum_i \hat{\alpha}_i (y_i - \hat{\mathbf{w}} \cdot \mathbf{x}_i) \Big/ \sum_i \alpha_i \tag{16.5.19}$$

Finally, the decision rule is $f(\mathbf{x}) = \hat{\mathbf{w}} \cdot \mathbf{x} + \hat{b}$.

A few observations will become important later:

- Data points with nonzero $\hat{\alpha}_i$ satisfy the constraints as equalities, i.e., they are support vectors.
- The only place that the data $\mathbf{x}_i$'s appear in (16.5.17) is in the Gram matrix $\mathbf{G}$.
- The only part of the calculation that scales with n (the dimensionality of the feature vector) is computing the components of the Gram matrix.
- All other parts of the calculation scale with m, the number of data points.

Thus, in going from primal to dual, we have substituted for a problem that scales (mostly) with the dimensionality of the feature matrix a problem that scales (mostly) with the number of data points. This might seem odd, because it makes problems with huge numbers of data points difficult. However, it makes easy, as we will soon see, problems with moderate amounts of data but *huge* feature vectors. This is in fact the regime where SVMs really shine.

16.5.4 The 1-Norm Soft-Margin SVM and Its Dual

The next important generalization is to relax the unrealistic assumption that there exists a hyperplane that separates the training data, i.e., get rid of the "oracle." We do this by introducing a so-called slack variable ξ_i for each data point $\mathbf{x}_i$. If the data point is one that *can* be separated by a fat plane, then $\xi_i = 0$. If it *can't* be, then $\xi_i > 0$ is the amount of the discrepancy, expressed by the modified inequality

$$y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1 - \xi_i \tag{16.5.20}$$

We must of course build in an inducement for the optimization to make the ξ_i's as small as possible, zero whenever possible. We thus have a trade-off between making the ξ_i's small and making the fat plane fat. In other words, we now have a problem that requires not only optimization, but also *regularization*, in the same sense as the discussion in §18.4. In the notation of equation (18.4.12), our quadratic forms ($\mathbf{w} \cdot \mathbf{w}$ or $\mathcal{L}$) are examples of $\mathcal{A}$'s. We need to invent a regularizing operator $\mathcal{B}$ that expresses our hopes for the ξ_i's, and then minimize $\mathcal{A} + \lambda\mathcal{B}$, instead of just $\mathcal{A}$ alone. As we vary λ in the range $0 < \lambda < \infty$, we explore a regularization trade-off curve.

The *1-norm soft-margin SVM* adopts, as the name indicates, a linear sum of the (positive) ξ_i's as its regularization operator. The primal problem is thus

$$\begin{array}{ll} \text{minimize:} & \frac{1}{2}\mathbf{w} \cdot \mathbf{w} + \lambda \sum_i \xi_i \\ \text{subject to:} & \xi_i \geq 0, \\ & y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1 - \xi_i \qquad i = 1, \ldots, m \end{array} \tag{16.5.21}$$

A possible variant is the *2-norm soft-margin SVM*, where the regularization term would be $\sum_i {\xi_i}^2$; however, this gives somewhat more complicated equations, so we will put it beyond our scope here.

Along the trade-off curve $0 < \lambda < \infty$, we vary from a a solution that prefers a *really fat* fat plane (no matter how many points are inside, or on the wrong side, of it) to a solution that is so miserly in allowing discrepancies that it settles for a fat plane with hardly any margin at all. The former is less accurate on the training data but possibly more robust on new data; the latter is as accurate as possible on the training data but possibly fragile (and less accurate) on new data. As in Chapter 19, the choice of λ is a design trade-off that you have to make. (However, we give you some guidance, below.)

Importantly, *any* nonnegative value of λ allows there to be *some* solution, whether the data are linearly separable or not. You can see this by noting that $\mathbf{w} = 0$ is always a feasible (but not optimal) solution of (16.5.21) for sufficiently large positive ξ_i's, no matter what the value of λ. If there is a feasible solution, there must, of course, be an optimal solution.

The very astute reader might notice that λ here seems to have the opposite qualitative sense from the λ's in Chapter 19. Specifically, $\lambda \to 0$ (here) gives the "softer," more robust, solution, while in Chapter 19 it is $\lambda \to \infty$ that, in a similar way, favors a priori smoothness. The reason for this switch is that the quadratic program (16.5.21) becomes the quadratic program (16.5.6) in the limit $\lambda \to \infty$, not 0. This is because there are no ξ_i's in the constraints in (16.5.6), so (16.5.21) must, in

the limit, force them to zero, requiring infinite λ. Correspondingly, as λ approaches zero, the ξ_i's become unconstrained. So the regularization term here indeed does act with the opposite sense from Chapter 19, because of the way it acts through the constraints, not the main functional.

Curiously, the dual to the 1-norm soft-margin SVM turns out to be *almost* identical to the dual of the (unrealistic) maximum margin SVM (16.5.17). Omitting details of the calculation, the result is

$$\begin{array}{ll} \text{minimize:} & \frac{1}{2}\boldsymbol{\alpha}\cdot\text{diag}(\mathbf{y})\cdot\mathbf{G}\cdot\text{diag}(\mathbf{y})\cdot\boldsymbol{\alpha}-\mathbf{e}\cdot\boldsymbol{\alpha} \\ \text{subject to:} & 0\le\alpha_i\le\lambda \qquad \text{for all } i \\ & \alpha\cdot\mathbf{y}=0 \end{array} \tag{16.5.22}$$

That is, the only difference is that there is now a constraining *upper* bound of λ on α_i in addition to the *lower* bound of zero. (This kind of constraint is called a *box constraint*.)

The formula for $\hat{\mathbf{w}}$ is unchanged from equation (16.5.13), while the Karush-Kuhn-Tucker conditions now become

$$\begin{aligned} (\hat{\alpha}_i-\lambda)\hat{\xi}_i &= 0 \\ \hat{\alpha}_i\left[y_i(\hat{\mathbf{w}}\cdot\mathbf{x}_i+\hat{b})-1+\hat{\xi}_i\right] &= 0 \end{aligned} \tag{16.5.23}$$

We see that, except for rare degenerate cases of double zeros,

$$\begin{array}{rcl} \hat{\alpha}_i=0 & \Longleftrightarrow & \text{data point } i \text{ on correct side of fat plane} \\ 0<\hat{\alpha}_i<\lambda & \Longleftrightarrow & \text{data point } i \text{ exactly on fat plane boundary (a support vector)} \\ \hat{\alpha}_i=\lambda & \Longleftrightarrow & \text{data point } i \text{ inside, or on wrong side, of fat plane} \end{array} \tag{16.5.24}$$

Here again we see that, as we reduce λ toward zero, pinning more and more α_i's at the value λ, we get solutions with increasing numbers of "wrong" points, but fatter fat planes.

The roundoff-averaged estimator for $\hat{b}$, analogous to equation (16.5.19), is

$$\hat{b}=\sum_i \hat{\alpha}_i(\lambda-\hat{\alpha}_i)(y_i-\hat{\mathbf{w}}\cdot\mathbf{x}_i)\Big/\sum_i \hat{\alpha}_i(\lambda-\hat{\alpha}_i) \tag{16.5.25}$$

Although the linear assumption (that is, using hyperplanes to separate the data) is still somewhat restrictive, the model defined by (16.5.22) does have some practical utility in problems where there is some reason to believe that the response is (at least somewhat) linear in the components of the feature vector. But that is far from the end of the story.

16.5.5 The Kernel Trick

Finally, we get to the generalization that gives SVMs their real power. Imagine an embedding function $\boldsymbol{\varphi}$ that maps n-dimensional feature vectors, in some manner, into a much higher N-dimensional space,

$$\mathbf{x}\quad(n\text{-dimensional})\quad\longrightarrow\quad\boldsymbol{\varphi}(\mathbf{x})\quad(N\text{-dimensional}) \tag{16.5.26}$$

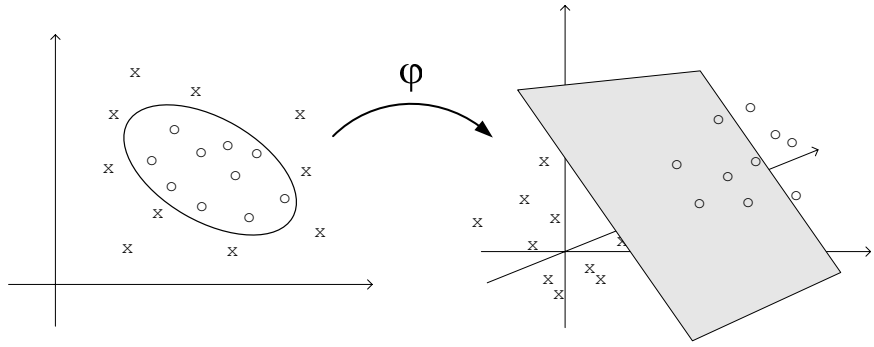

Figure 16.5.2. When feature vectors are mapped from a lower-dimensional space (here 2) to a higher-dimensional embedding space (here 3), nonlinear separation surfaces can become well approximated by linear ones. In practice, *very* high-dimensional embedding spaces are used, but they enter the SVM calculation only implicitly, through the "kernel trick."

The basic idea, as shown in Figure 16.5.2, is that a very nonlinear separating surface in the n-dimensional space might map into (or be well approximated by) a linear hyperplane in the N-dimensional space.

To see why this might work, consider this mapping from two to five dimensions:

$$(x_0, x_1) \quad \xrightarrow{\varphi} \quad (x_0^2, x_0 x_1, x_1^2, x_0, x_1) \tag{16.5.27}$$

With this mapping, a decision rule $f(\mathbf{x})$ that is constructed as linear in the embedding space becomes general enough to include all linear and quadratic forms (lines, ellipses, hyperbolas) in the original feature space, namely,

$$f(\mathbf{x}) = F[\boldsymbol{\varphi}(\mathbf{x})] \equiv \mathbf{W} \cdot \boldsymbol{\varphi}(\mathbf{x}) + B \tag{16.5.28}$$

where we are using uppercase letters for quantities in the embedding space. Although $N = 5$ in this example, it might instead have a value like a million or a billion (we'll see how this works in a minute).

Give our data, how do we find $\mathbf{W}$ and B in the embedding space? Let's try exactly as before, but just in the higher-dimensional space. The primal problem (compare to equation 16.5.21) is

$$\begin{array}{ll} \text{minimize:} & \frac{1}{2}\mathbf{W} \cdot \mathbf{W} + \lambda \sum_i \Xi_i \\ \text{subject to:} & \Xi_i \geq 0, \\ & y_i(\mathbf{W} \cdot \boldsymbol{\varphi}(\mathbf{x}_i) + B) \geq 1 - \Xi_i \qquad i = 1, \ldots, m \end{array} \tag{16.5.29}$$

Uh-oh! This is a quadratic programming problem in a million- or billion-dimensional space, not likely to be tractable on your ordinary desktop computer.

What about the dual problem? It turns out to be

$$\begin{array}{ll} \text{minimize:} & \frac{1}{2}\boldsymbol{\alpha} \cdot \text{diag}(\mathbf{y}) \cdot \mathbf{K} \cdot \text{diag}(\mathbf{y}) \cdot \boldsymbol{\alpha} - \mathbf{e} \cdot \boldsymbol{\alpha} \\ \text{subject to:} & 0 \leq \alpha_i \leq \lambda \qquad \text{for all } i \\ & \alpha \cdot \mathbf{y} = 0 \end{array} \tag{16.5.30}$$

This is exactly the same as (16.5.22), except that the Gram matrix G_{ij} has been replaced by the so-called *kernel* K_{ij},

$$K_{ij} \equiv K(\mathbf{x}_i, \mathbf{x}_j) \equiv \boldsymbol{\varphi}(\mathbf{x}_i) \cdot \boldsymbol{\varphi}(\mathbf{x}_j) \tag{16.5.31}$$

Well, this is progress. The quadratic programming problem (16.5.30) is no harder than the original problem (16.5.22)! Both live in a space of dimension m, the number of data points, and both get fed a fixed matrix, precalculated from the data: G_{ij} in one case, K_{ij} in the other.

We have succeeded in maneuvering the "curse of dimensionality" into a very tight corner, namely the calculation of just the m^2 values K_{ij}. Now we annihilate it entirely with *the kernel trick*:

The "trick" is that we never really need to know the mapping $\boldsymbol{\varphi}(\mathbf{x})$ at all. All we need is a way of computing a kernel K_{ij} that *could* have come from some mapping $\boldsymbol{\varphi}(\mathbf{x})$, that is, a matrix of size $m \times m$ with the mathematical properties of an inner product space in higher dimension. We already know one possible kernel, namely the Gram matrix G_{ij}. Here are some provable properties of kernel functions $K(\mathbf{x}_i, \mathbf{x}_j)$ in general:

- $K_{ij} = K(\mathbf{x}_i, \mathbf{x}_j)$ must be symmetric in i and j and must have nonnegative eigenvalues (Mercer's theorem).
- Any multinomial combination of kernel functions is a kernel function. That is, you can freely combine kernel functions by multiplication, addition, and scaling by a constant.
- $K(\boldsymbol{\varphi}(\mathbf{x}_i), \boldsymbol{\varphi}(\mathbf{x}_j))$ is a kernel if $K(,)$ is one, for any $\boldsymbol{\varphi}$. This generalizes the original idea of the embedding space.
- $K(\mathbf{x}_i, \mathbf{x}_j) = g(\mathbf{x}_i) g(\mathbf{x}_j)$ is always a kernel, for any function g.

Once you settle on a kernel and solve the quadratic programming problem (16.5.30), then your final decision rule for any new feature vector $\mathbf{x}$ is

$$f(\mathbf{x}) = \sum_i \hat{\alpha}_i y_i K(\mathbf{x}_i, \mathbf{x}) + \hat{b} \tag{16.5.32}$$

where (again using the averaging trick)

$$\hat{b} = \sum_i \hat{\alpha}_i (\lambda - \hat{\alpha}_i)[y_i - \sum_j \hat{\alpha}_i y_j K(\mathbf{x}_j, \mathbf{x}_i)] \Big/ \sum_i \hat{\alpha}_i (\lambda - \hat{\alpha}_i) \tag{16.5.33}$$

While the construction of the ideal kernel for any particular problem can involve some art, some very generic kernels turn out to be quite powerful in solving real-world problems. Often you can just try a few of these and pick the one that works best. The following are good ones to try first:

$$\begin{aligned}
\text{linear:} \qquad & K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i \cdot \mathbf{x}_j \\
\text{power:} \qquad & K(\mathbf{x}_i, \mathbf{x}_j) = (\mathbf{x}_i \cdot \mathbf{x}_j)^d, \qquad 2 \le d \le 20 \text{ (say)} \\
\text{polynomial:} \qquad & K(\mathbf{x}_i, \mathbf{x}_j) = (a\, \mathbf{x}_i \cdot \mathbf{x}_j + b)^d \\
\text{sigmoid:} \qquad & K(\mathbf{x}_i, \mathbf{x}_j) = \tanh(a\, \mathbf{x}_i \cdot \mathbf{x}_j + b) \\
\text{Gaussian radial basis function:} \qquad & K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\tfrac{1}{2}|\mathbf{x}_i - \mathbf{x}_j|^2/\sigma^2)
\end{aligned} \tag{16.5.34}$$

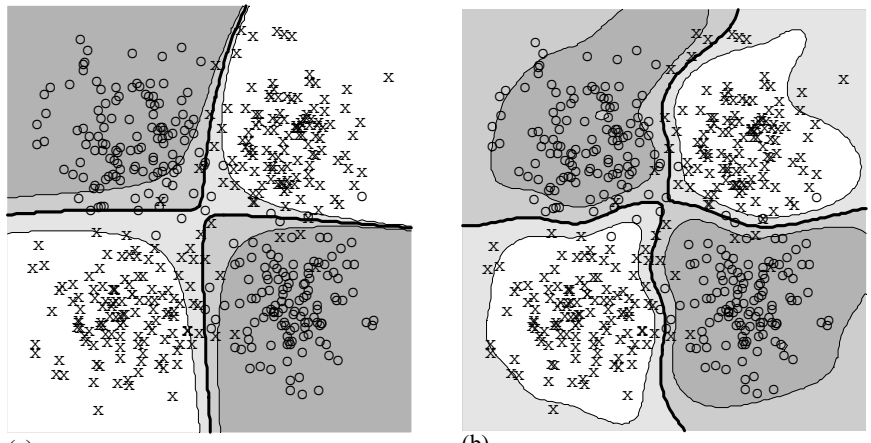

Figure 16.5.3. SVMs that "learn" to partition the plane. The input data are drawn from four two-dimensional Gaussians, slightly overlapping, with diagonally opposite ones given the same label (x or o). Heavy solid lines are the decision rule surfaces $f(\mathbf{x}) = 0$ derived by the SVMs. Lighter lines show $f(\mathbf{x}) = \pm 1$. (a) Polynomial kernel with $d = 8$. (b) Gaussian radial basis function kernel.

See §2.3 of [5] for additional standard kernels. Chapter 13 of [5] describes many specialized kernels, e.g., for comparing strings or passages of text, for image recognition, and for a number of other applications.

Figure 16.5.3 shows a test example using both a polynomial kernel with $d = 8$ and a Gaussian radial basis kernel. It is characteristic of the Gaussian kernel that it is more influenced by local nearest neighbor effects (which may be good or bad, depending on the application), while polynomial kernels seek smoother, more global solutions.

Although it is beyond our scope in this section, we should mention that the kernel trick is applicable not only to SVMs (that is, to algorithms based on separating hyperplanes), but also to a number of other pattern recognition algorithms, for example principal component analysis (PCA) and the Fisher discriminant algorithm. See [5] and [6] for extensive treatments of these *kernel-based learning algorithms*.

16.5.6 Some Practical Advice on SVMs

The Gaussian radial basis function kernel is very popular, because it has only one adjustable parameter, σ, and it is easy to guess a first value to try, namely any characteristic distance between nearby points in the feature space. As mentioned, the Gaussian kernel classifies to some degree by the local neighborhood consensus.

For the polynomial kernels, start by choosing a and b to make $a\,\mathbf{x}_i \cdot \mathbf{x}_j + b$ lie between ± 1 for all i and j. The power d has a (very rough) interpretation as how many different features you want the comparison to "mix." That is, $d = 1$ (linear) partitions the space by one feature at at time; $d = 2$ looks at pairs of features simultaneously, and so on. Also very roughly, the difference between power and polynomial is whether you want to consider only exactly d features at a time (power), or all combinations of d or fewer features (polynomial). These interpretations should not be taken too seriously, however. Specifically, larger d is not always better.

We have not said much about how to choose λ, the regularization parameter.

Try $\lambda = 1$ first, then try increasing and decreasing it by factors of 10. There is typically a broad plateau, as a function of $\log_{10}(\lambda)$, where the precise value of λ doesn't matter much. There is some belief that $\langle \mathbf{x}_i \cdot \mathbf{x}_j \rangle^{-1}$ or $\langle K(\mathbf{x}_i, \mathbf{x}_j) \rangle^{-1}$, where angle brackets denote averages over all i, j pairs, are good starting guesses; but for properly scaled kernels these should not be too different from unity in any case.

As you vary λ and repeatedly solve the quadratic program, look at the fraction of α_i's that are pinned at zero, pinned at λ, or floating between these limits. A good profile will often have the biggest fraction at zero, a smaller (but not necessarily much smaller) fraction at λ, and the smallest fraction in between (see equation 16.5.24 for interpretation). These fractions are also often good indicators for adjusting parameters in your kernel. Naturally you will also be looking at the fraction of your training data that is predicted correctly, that is, has $y_i f(\mathbf{x}_i) > 0$.

Below, we will give a short, self-contained program for finding the solution to SVMs; but for anything other than small problems you will want to use a more sophisticated software package. There are many tricks and shortcuts that can speed the solution of an SVM relative to the general problem of quadratic programming — good SVM packages take advantage of these. For example, a good package should take advantage of sparseness in the feature vectors to save on computation. Our favorite package is Thorsten Joachims' *SVMlight* [7], available for free on the Web. *Gist* [8] is another popular free implementation. The Web site cited in [2] has a page with links to a wide variety of SVM software.

16.5.7 The Mangasarian-Musicant Variant and Its Solution by SOR

Mangasarian and Musicant [9,10] have suggested a very slight variant of equation (16.5.21), and its kernel generalization, that has the interesting property that it can be solved, quite compactly, by the method of successive overrelaxation (SOR; see §20.5.1). In particular, a complete SVM solution program using SOR is less than 100 lines long. We discuss this *M-M variant* here, and implement it in code, just because of its brevity. We have used this code for problems of up to several thousand data points, with feature vectors of length several hundred. Such problems take seconds to solve on a desktop machine. For larger problems, our advice is that you use the more efficient specialized packages [7,8]. *SVMlight*, for example, is typically about an order of magnitude faster than the code we give below.

The primal problem in the 1-norm soft-margin form of the M-M variant is

$$\begin{aligned} &\text{minimize:} && \tfrac{1}{2}\mathbf{w}\cdot\mathbf{w} + b^2 + \lambda \sum_i \xi_i \\ &\text{subject to:} && \xi_i \geq 0, \\ & && y_i(\mathbf{w}\cdot\mathbf{x}_i + b) \geq 1 - \xi_i \qquad i = 1, \ldots, m \end{aligned} \tag{16.5.35}$$

The only difference from (16.5.21) is that a term b^2 has been added to the functional that is minimized. On its face, this should have the effect of slightly favoring hyperplanes closer to the origin, all else being equal, an innocuous (albeit arbitrary) change. The real purpose of the b^2 term, however, is its algebraic effect when we calculate the dual problem:

$$\boxed{\begin{array}{ll}\text{minimize:} & \frac{1}{2}\boldsymbol{\alpha}\cdot\text{diag}(\mathbf{y})\cdot(\mathbf{G}+\mathbf{e}\otimes\mathbf{e})\cdot\text{diag}(\mathbf{y})\cdot\boldsymbol{\alpha}-\mathbf{e}\cdot\boldsymbol{\alpha} \\ \text{subject to:} & 0\le\alpha_i\le\lambda \qquad \text{for all } i\end{array}} \tag{16.5.36}$$

Aside from an extra term $\mathbf{e}\otimes\mathbf{e}$ (the matrix of all ones) now added to the Gram matrix, the main change from (16.5.21) is that the equality constraint is gone! This renders the solution *much* more tractable numerically. The dual problem also now has a simpler expression for $\hat{b}$,

$$\hat{b}=\sum_i \hat{\alpha}_i y_i \tag{16.5.37}$$

(As before, $\hat{\mathbf{w}}$ is computed from 16.5.13.)

When we do the kernel trick, the only change in (16.5.36) is to change G_{ij} to K_{ij}. Equation (16.5.37) still holds, but $\hat{b}$ is actually superfluous since the decision rule can be written directly as

$$f(\mathbf{x})=\sum_i \hat{\alpha}_i y_i[K(\mathbf{x}_i,\mathbf{x})+1] \tag{16.5.38}$$

Mangasarian and Musicant have shown that the solution of the M-M variant SVM is often identical to the solution of the the standard 1-norm soft-margin SVM (albeit with a different value of λ) and is almost never significantly different. What is quite different, however, is that (16.5.36) and its kernel version can be solved by the following, linearly convergent, relaxation procedure:

- Define $\mathbf{M}\equiv\text{diag}(\mathbf{y})\cdot(\mathbf{K}+\mathbf{e}\otimes\mathbf{e})\cdot\text{diag}(\mathbf{y})$.
- Initialize all the α_i's to zero.
- Repeat *ad libitum* the relaxation replacement, for $i=1,2,\ldots,m$,

$$\alpha_i \leftarrow \mathcal{P}\left[\alpha_i-\omega\frac{1}{M_{ii}}\left(\sum_j M_{ij}\alpha_j-1\right)\right] \tag{16.5.39}$$

Here $\mathcal{P}$ is the projection operator that just puts α back into its allowed range. [Note the similarity to the method of projection onto convex sets (POCS) in §19.5.2.]

$$\mathcal{P}=\begin{cases}0, & \alpha<0\\ \alpha, & 0\le\alpha\le\lambda\\ \lambda, & \alpha>\lambda\end{cases} \tag{16.5.40}$$

The constant ω is the overrelaxation parameter, exactly as in §20.5.1. You pick it in the range $0<\omega<2$. In our experience, the convergence rate does not depend sensitively on ω. If you don't have a better idea, take $\omega=1.3$.

Our implementation begins with a virtual class that defines the interface to a kernel function,

svm.h
```
struct Svmgenkernel {
```
Virtual class that defines what a kernel structure needs to provide.

`Int m, kcalls;`	No. of data points; counter for kernel calls.
`MatDoub ker;`	Locally stored kernel matrix.
`VecDoub_I &y;`	Must provide reference to the y_i's.
`MatDoub_I &data;`	Must provide reference to the $\mathbf{x}_i$'s.

```
    Svmgenkernel(VecDoub_I &yy, MatDoub_I &ddata)
        : m(yy.size()),kcalls(0),ker(m,m),y(yy),data(ddata) {}
    Every kernel structure must provide a kernel function that returns the kernel for arbitrary
    feature vectors.
    virtual Doub kernel(const Doub *xi, const Doub *xj) = 0;
    inline Doub kernel(Int i, Doub *xj) {return kernel(&data[i][0],xj);}
    Every kernel structure's constructor must call fill to fill the ker matrix.
    void fill() {
        Int i,j;
        for (i=0;i<m;i++) for (j=0;j<=i;j++) {
            ker[i][j] = ker[j][i] = kernel(&data[i][0],&data[j][0]);
        }
    }
};
```

Basically, a kernel structure is required to provide references to the data (the $\mathbf{x}_i$'s) and the $\mathbf{y}_i$'s, a matrix of kernel values for all pairs of data points, and two forms of the kernel function: one with two arbitrary feature vectors as arguments, and another with one argument a data point and the other an arbitrary feature vector. Here are three examples of kernels, for three of the standard kernels in equation (16.5.34), built on the above Svmgenkernel.

svm.h

```
struct Svmlinkernel : Svmgenkernel {
```
Kernel structure for the linear kernel, the dot product of two feature vectors (with overall means of each component subtracted).
```
    Int n;
    VecDoub mu;
    Svmlinkernel(MatDoub_I &ddata, VecDoub_I &yy)
```
Constructor is called with the $m \times n$ data matrix, and the vector of y_i's, length m.
```
        : Svmgenkernel(yy,ddata), n(data.ncols()), mu(n) {
        Int i,j;
        for (j=0;j<n;j++) mu[j] = 0.;
        for (i=0;i<m;i++) for (j=0;j<n;j++) mu[j] += data[i][j];
        for (j=0;j<n;j++) mu[j] /= m;
        fill();
    }
    Doub kernel(const Doub *xi, const Doub *xj) {
        Doub dott = 0.;
        for (Int k=0; k<n; k++) dott += (xi[k]-mu[k])*(xj[k]-mu[k]);
        return dott;
    }
};

struct Svmpolykernel : Svmgenkernel {
```
Kernel structure for the polynomial kernel.
```
    Int n;
    Doub a, b, d;
    Svmpolykernel(MatDoub_I &ddata, VecDoub_I &yy, Doub aa, Doub bb, Doub dd)
```
Constructor is called with the $m \times n$ data matrix, the vector of y_i's, length m, and the constants a, b, and d.
```
        : Svmgenkernel(yy,ddata), n(data.ncols()), a(aa), b(bb), d(dd) {fill();}
    Doub kernel(const Doub *xi, const Doub *xj) {
        Doub dott = 0.;
        for (Int k=0; k<n; k++) dott += xi[k]*xj[k];
        return pow(a*dott+b,d);
    }
};

struct Svmgausskernel : Svmgenkernel {
```
Kernel structure for the Gaussian radial basis function kernel.
```
    Int n;
```

```
    Doub sigma;
    Svmgausskernel(MatDoub_I &ddata, VecDoub_I &yy, Doub ssigma)
    Constructor is called with the m × n data matrix, the vector of y_i's, length m, and the
    constant σ.
        : Svmgenkernel(yy,ddata), n(data.ncols()), sigma(ssigma) {fill();}
    Doub kernel(const Doub *xi, const Doub *xj) {
        Doub dott = 0.;
        for (Int k=0; k<n; k++) dott += SQR(xi[k]-xj[k]);
        return exp(-0.5*dott/(sigma*sigma));
    }
};
```

The above is all prefatory to the SVM solution structure. You declare an instance of `Svm` with your kernel as the argument. It then makes available three functions: `relax` performs one "group" of relaxation steps and returns the norm of how much change in $\boldsymbol{\alpha}$ has occurred. (We define "group" below.) You call `relax` repeatedly, with λ and ω as arguments, until the returned value is small enough: 10^{-3} or 10^{-4} is usually plenty. Then (and only then) you may repeatedly call either of two forms of `predict`, which returns the decision rule $f(\mathbf{x})$. One form of `predict` returns the prediction for data points, the other for arbitrary new feature vectors. If you want to examine the α_i's, or count how many are pinned at 0 or λ, you can examine the vector `alph`.

svm.h

```
struct Svm {
```

Class for solving SVM problems by the SOR method.

```
    Svmgenkernel &gker;                     Reference bound to user's kernel (and data).
    Int m, fnz, fub, niter;
    VecDoub alph, alphold;                  Vectors of α's before and after a step.
    Ran ran;                                Random number generator.
    Bool alphinit;
    Doub dalph;                             Change in norm of the α's in one step.
    Svm(Svmgenkernel &inker) : gker(inker), m(gker.y.size()),
        alph(m), alphold(m), ran(21), alphinit(false) {}
        Constructor binds the user's kernel and allocates storage.
    Doub relax(Doub lambda, Doub om) {
    Perform one group of relaxation steps: a single step over all the α's, and multiple steps
    over only the interior α's.
        Int iter,j,jj,k,kk;
        Doub sum;                           Index when α's are sorted by value.
        VecDoub pinsum(m);                  Stored sums over noninterior variables.
        if (alphinit == false) {            Start all α's at 0.
            for (j=0; j<m; j++) alph[j] = 0.;
            alphinit = true;
        }
        alphold = alph;                     Save old α's.
        Here begins the relaxation pass over all the α's.
        Indexx x(alph);                     Sort α's, then find first nonzero one.
        for (fnz=0; fnz<m; fnz++) if (alph[x.indx[fnz]] != 0.) break;
        for (j=fnz; j<m-2; j++) {               Randomly permute all the nonzero α's.
            k = j + (ran.int32() % (m-j));
            SWAP(x.indx[j],x.indx[k]);
        }
        for (jj=0; jj<m; jj++) {                Main loop over α's.
            j = x.indx[jj];
            sum = 0.;
            for (kk=fnz; kk<m; kk++) {          Sums start with first nonzero.
                k = x.indx[kk];
                sum += (gker.ker[j][k] + 1.)*gker.y[k]*alph[k];
            }
            alph[j] = alph[j] - (om/(gker.ker[j][j]+1.))*(gker.y[j]*sum-1.);
```

```
            alph[j] = MAX(0.,MIN(lambda,alph[j]));          Projection operator.
            if (jj < fnz && alph[j]) SWAP(x.indx[--fnz],x.indx[jj]);
        }                                   (Above) Make an α active if it becomes nonzero.
        Here begins the relaxation passes over the interior α's.
        Indexx y(alph);                                     Sort. Identify interior α's.
        for (fnz=0; fnz<m; fnz++) if (alph[y.indx[fnz]] != 0.) break;
        for (fub=fnz; fub<m; fub++) if (alph[y.indx[fub]] == lambda) break;
        for (j=fnz; j<fub-2; j++) {                         Permute.
            k = j + (ran.int32() % (fub-j));
            SWAP(y.indx[j],y.indx[k]);
        }
        for (jj=fnz; jj<fub; jj++) {                        Compute sums over pinned α's just
            j = y.indx[jj];                                     once.
            sum = 0.;
            for (kk=fub; kk<m; kk++) {
                k = y.indx[kk];
                sum += (gker.ker[j][k] + 1.)*gker.y[k]*alph[k];
            }
            pinsum[jj] = sum;
        }
        niter = MAX(Int(0.5*(m+1.0)*(m-fnz+1.0)/(SQR(fub-fnz+1.0))),1);
        Calculate a number of iterations that will take about half as long as the full pass just
        completed.
        for (iter=0; iter<niter; iter++) {                  Main loop over α's.
            for (jj=fnz; jj<fub; jj++) {
                j = y.indx[jj];
                sum = pinsum[jj];
                for (kk=fnz; kk<fub; kk++) {
                    k = y.indx[kk];
                    sum += (gker.ker[j][k] + 1.)*gker.y[k]*alph[k];
                }
                alph[j] = alph[j] - (om/(gker.ker[j][j]+1.))*(gker.y[j]*sum-1.);
                alph[j] = MAX(0.,MIN(lambda,alph[j]));
            }
        }
        dalph = 0.;                                         Return change in norm of α vector.
        for (j=0;j<m;j++) dalph += SQR(alph[j]-alphold[j]);
        return sqrt(dalph);
    }
    Doub predict(Int k) {
    Call only after convergence via repeated calls to relax. Returns the decision rule f(x) for
    data point k.
        Doub sum = 0.;
        for (Int j=0; j<m; j++) sum += alph[j]*gker.y[j]*(gker.ker[j][k]+1.0);
        return sum;
    }
    Doub predict(Doub *x) {
    Call only after convergence via repeated calls to relax. Returns the decision rule f(x) for
    an arbitrary feature vector.
        Doub sum = 0.;
        for (Int j=0; j<m; j++) sum += alph[j]*gker.y[j]*(gker.kernel(j,x)+1.0);
        return sum;
    }
};
```

Although the enforced brevity doesn't allow for *too* many optimizing tricks, `Svm` does have a couple that bear mentioning:

First, each call to the `relax` routine performs, as previously mentioned, a group of relaxations. Specifically, it does one full relaxation pass over all the α_i's, and then multiple passes over only the "interior" α_i's, i.e., those that are not pinned at either 0 or λ. These passes are typically *much* faster than the full pass, since most variables

are typically pinned. To realize the gain, sums over pinned variables that don't vary are computed only once at the beginning of these multiple passes. The number of such passes is calculated dynamically so as to take about half as long as the full pass just taken.

Second, before each pass (both the full and interior), the order of the variables is randomized by a permutation generated from a `Ran` object (§7.1). This randomization speeds up the convergence by as much as an order of magnitude.

CITED REFERENCES AND FURTHER READING:

Boser, B.E., Guyon, I.M., and Vapnik, V.N. 1992, in D. Haussler, ed., *Proceedings of the 5th Annual ACM Workshop on Computational Learning Theory* (New York: ACM Press).[1]

Christianini, N. and Shawe-Taylor, J. 2000, *An Introduction to Support Vector Machines* (Cambridge, U.K.: Cambridge University Press); related Web site `http://www.support-vector.net`.[2]

Bazaraa, M.S., Sherali, H.D., and Shetty, C.M. 2006, *Nonlinear Programming: Theory and Algorithms*, 3rd ed. (Hoboken, NJ: Wiley).[3]

den Hertog, D. 1994, *Interior Point Approach to Linear, Quadratic and Convex Programming: Algorithms and Complexity* (Dordrecht: Kluwer).[4]

Schölkopf, B. and Smola, A.J. 2002, *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond* (Cambridge, MA: MIT Press).[5]

Shawe-Taylor, J. and Christianini, N. 2004, *Kernel Methods for Pattern Analysis* (Cambridge, UK: Cambridge University Press).[6]

Vapnik, V. 1998, *Statistical Learning Theory* (New York: Wiley).

Joachims, T. 1999–, *SVMlight, Implementing Support Vector Machines in C*, at `http://svmlight.joachims.org`.[7]

Noble, W.S. and Pavlidis, P. 1999–, *Gist: Software Tools for Support Vector Machine Classification and for Kernel Principal Components Analysis*, at `http://microarray.cpmc.columbia.edu/gist`.[8]

Mangasarian, O.L. and Musicant, D.R. 1999, "Successive Overrelaxation for Support Vector Machines," *IEEE Transactions on Neural Networks*, vol. 10, no. 5, p. 1032.[9]

Mangasarian, O.L. and Musicant, D.R. 2001, in *Complementarity: Applications, Algorithms and Extensions*, M.C. Ferris, O.L. Mangasarian and J.-S. Pang, eds. (Dordrecht: Kluwer) pp. 233-251.[10]